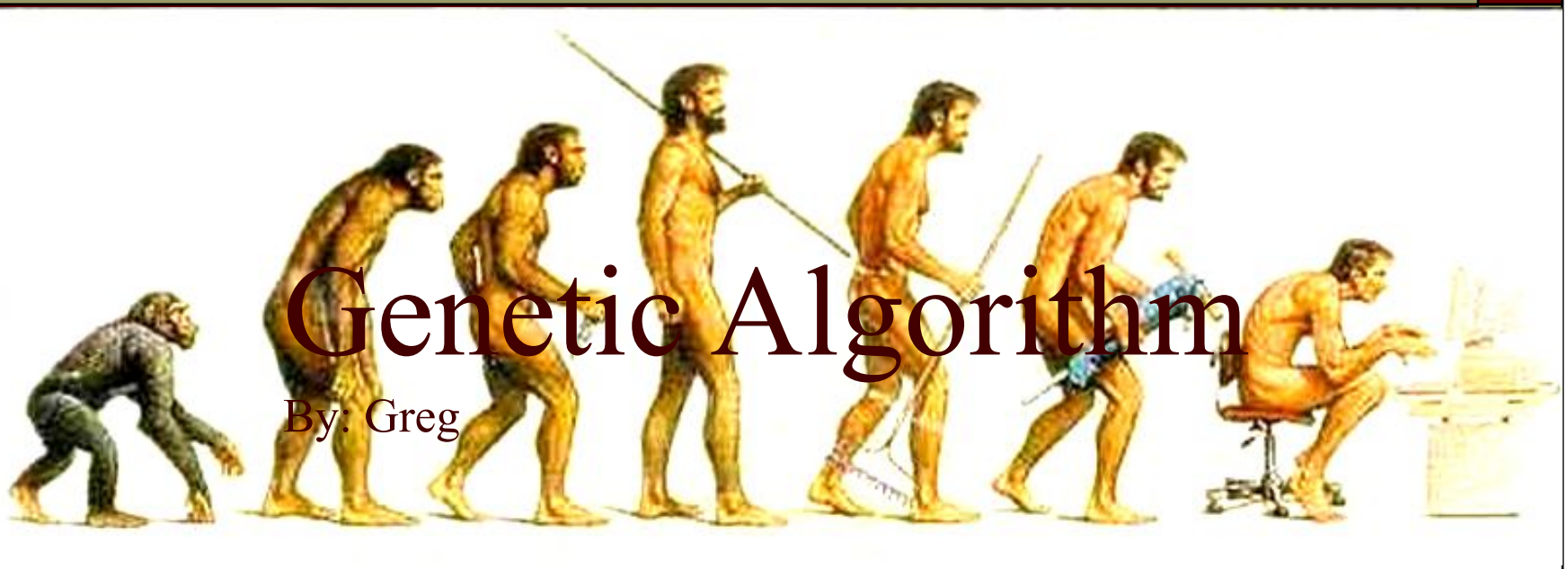


Genetic Algorithm

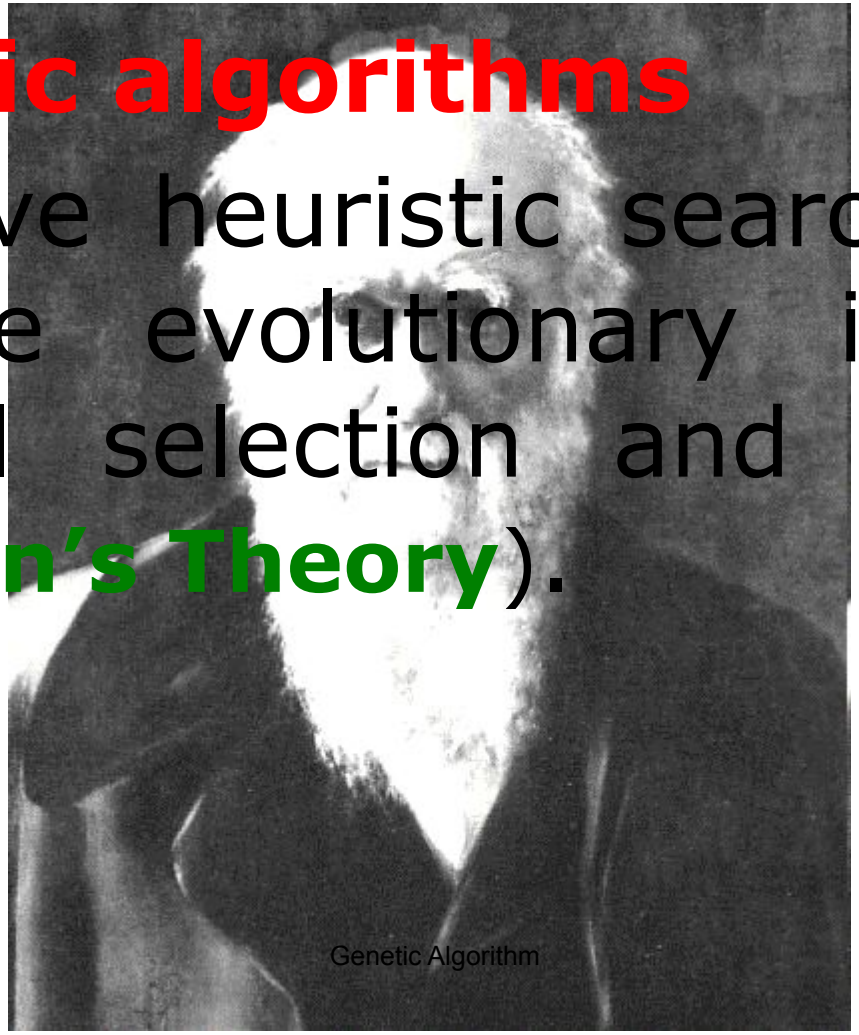
By: Greg

- What is a Genetic Algorithm?
- GA Life Cycle
  - Crossover
  - Mutation
  - Selection
  - Fitness

What is a Genetic Algorithm?

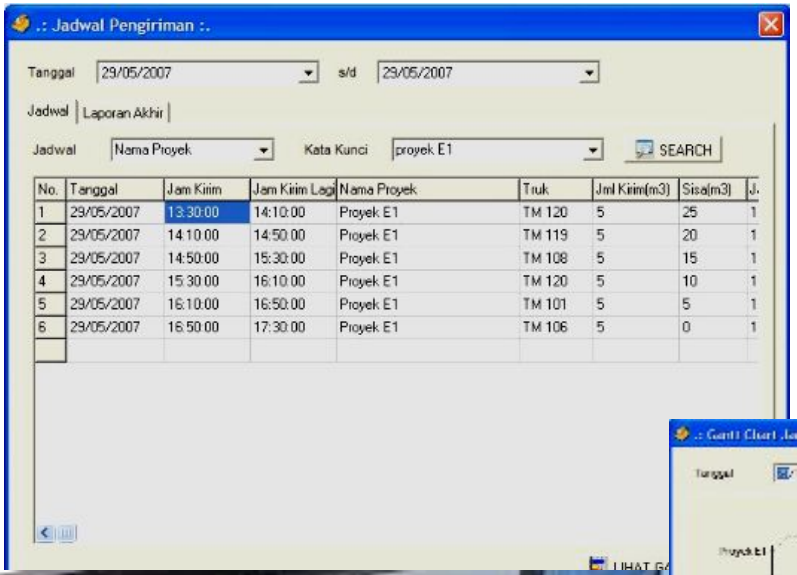
- **Genetic algorithms**

  Adaptive heuristic search based on the evolutionary ideas of natural selection and genetics (**Darwin's Theory**).

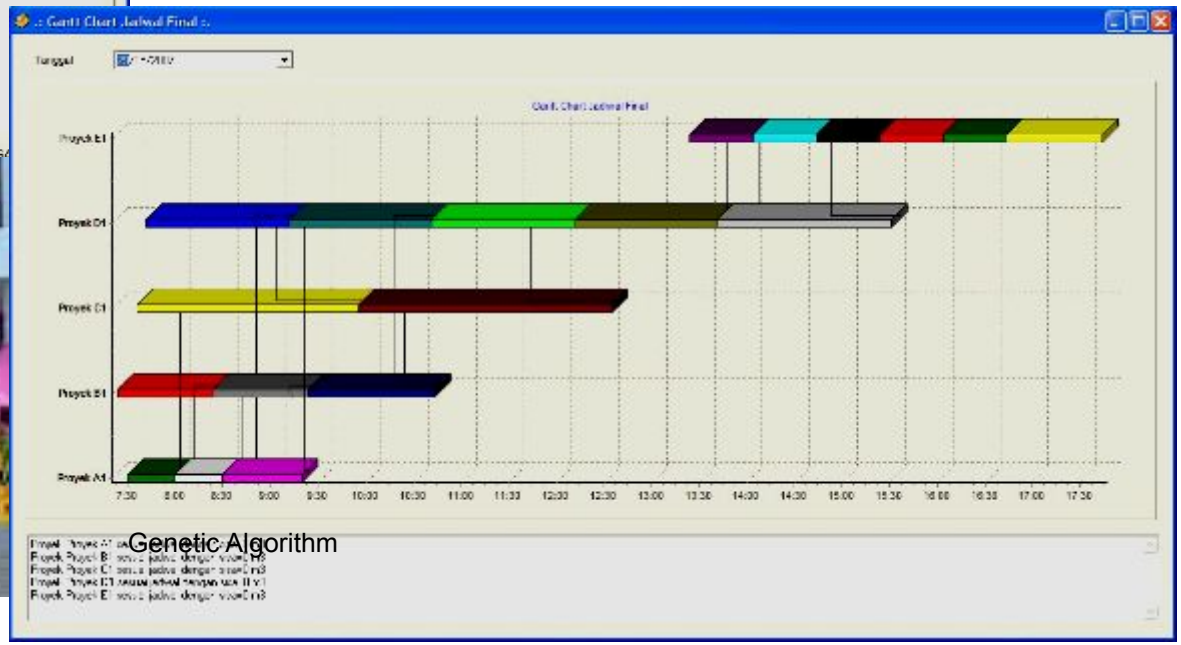


Aplication Examples

Ready-mix Concrete Delivery Scheduling Optimization **(Vince Pongawa)**



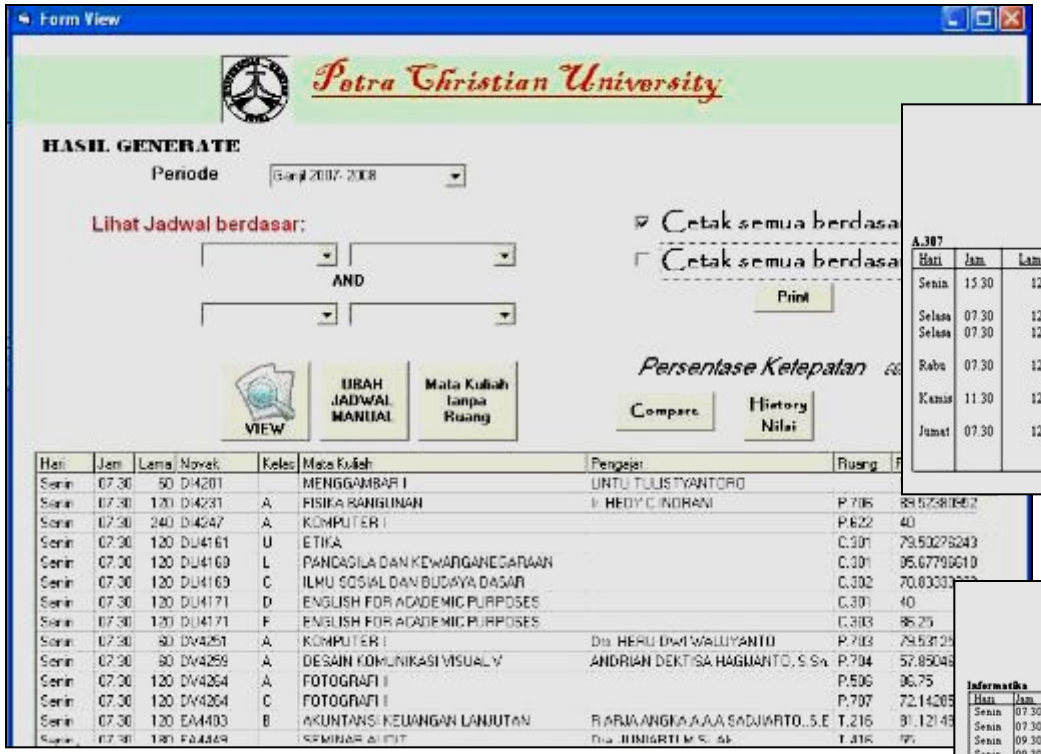
Budhi GS, Sundoro D, Pongawa V, (2008), Penggunaan Compact Genetic Algorithm (cGA) Untuk Optimasi Penjadwalan Pengiriman Beton Ready-Mix, Proceeding Konferensi Nasional Sistem dan Informatika 2008; Bali, November 15, 2008

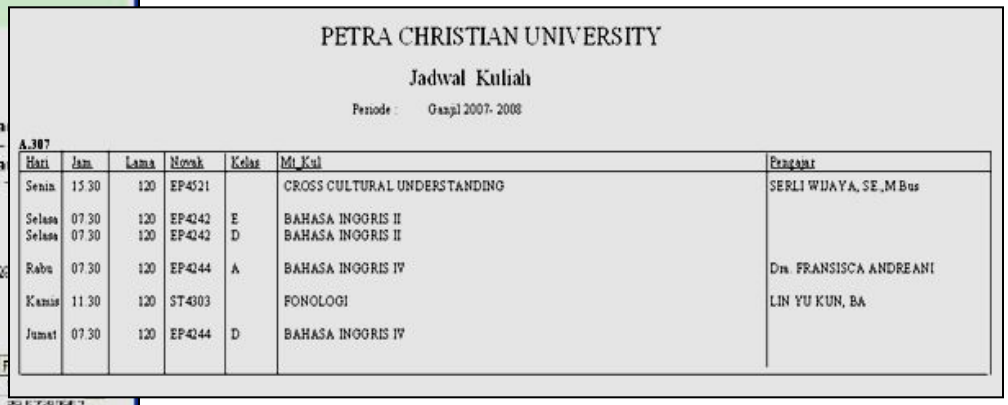


Genetic Algorithm

Aplication Examples

Classroom schedulling in U.K. Petra using compact GA





PETRA CHRISTIAN UNIVERSITY

Jadwal Kuliah

Periode : Ganjil 2007- 2008

A.307

Hari	Jam	Lama	Novak	Kelas	Mt Kul	Pengajar
Senin	15.30	120	EP4521		CROSS CULTURAL UNDERSTANDING	SERLI WIJAYA, SE.,M Bus
Selasa	07.30	120	EP4242	E	BAHASA INGGRIS II	
Selasa	07.30	120	EP4242	D	BAHASA INGGRIS II	
Rabu	07.30	120	EP4244	A	BAHASA INGGRIS IV	Drs. FRANSISCA ANDREANI
Kamis	11.30	120	ST4303		FONOLOGI	LIN YU KUN, BA
Jumat	07.30	120	EP4244	D	BAHASA INGGRIS IV	

PETRA CHRISTIAN UNIVERSITY

JADWAL KULIAH

Periode : Ganjil 2007- 2008

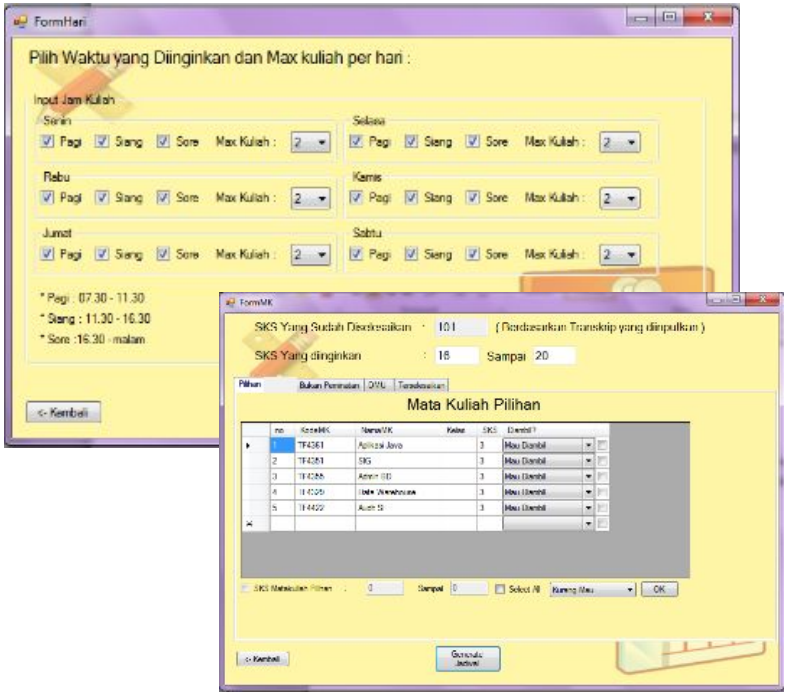
Informatika

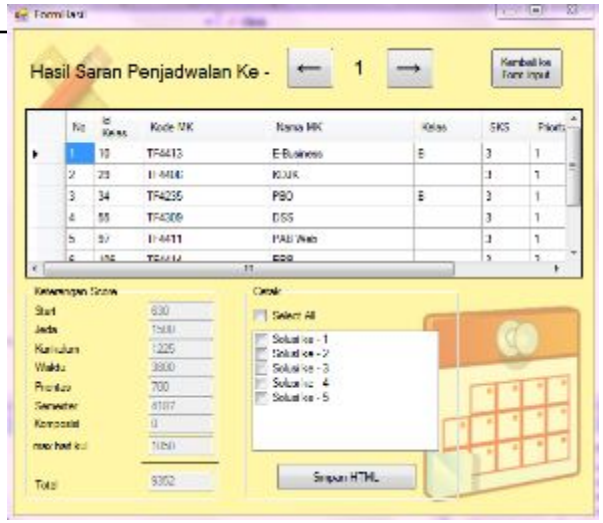
Hari	Jam	Lama	Novak	Kelas	Mt Kul	Pengajar	Ruang
Senin	07.30	180	TF4285		PENGOLAHAN CITRA DIGITAL	Ir. KARTIKA GUNADI, M.T.	1.308A
Senin	07.30	120	TF4211	A	KALKULUS I		P.205
Senin	09.30	120	TF4218	C	PENGANTAR MANAJEMEN	AGOSTINUS NOER TJAHYANA, S.Kom.,MMT	P.203
Senin	09.30	120	TF4269		DESAIN DAN ANALISA ALGORITMA	GREGORIUS SATIA BUDHI, ST.,MT	P.203
Senin	09.30	120	TF4219	C	STRUKTUR DATA		P.204
Senin	09.30	120	TF4211	A	KALKULUS I		P.707
Senin	13.30	120	TF4333		DATA MINING	GREGORIUS SATIA BUDHI, ST.,MT	P.504
Senin	13.30	120	TF4211	D	KALKULUS I		P.503
Senin	13.30	120	TF4245	A	MATEMATIKA DISKRIT	FELICIA SOEDJIANTO, S.Kom	P.617
Senin	14.30	180	TF4353		MANAJEMEN KEAMANAN KOMPUTER	AGOSTINUS NOER TJAHYANA, S.Kom.,MMT	P.627
Senin	15.30	180	TF4241	A	SISTEM INFORMASI	Ir. DJONI HARYADI S., M.Eng	P.430
Senin	15.30	120	TF4245	B	MATEMATIKA DISKRIT	ANDREAS HANDOJO, S.T.,MMT	1.407
Senin	15.30	120	TF4211	D	KALKULUS I		1.308B
Senin	15.30	180	TF4363		PEMROGRAMAN OPEN SOURCE	JUSTINUS ANDJARWIRAWAN, S.T.,M.Sc.	P.205
Senin	15.30	120	TF4204	B	ALGORITMA DAN PEMROGRAMAN	RUDY ADIPRANATA, S.T.,M.Eng	P.205
Senin	15.30	180	TF4235	B	PEMROGRAMAN BERORIENTASI OBYEK	OVILIANI YENTI YULIANA, ST.,MSCIS	P.804A
Selasa	07.30	120	TF4204	A	ALGORITMA DAN PEMROGRAMAN		P.219
Selasa	07.30	120	TF4204	C	ALGORITMA DAN PEMROGRAMAN	Ir. KARTIKA GUNADI, M.T.	P.707
Selasa	08.30	120	TF4229	A	BASIS DATA	SILVIA ROSTIANINGSIH, S.Kom.MMT.	P.620
Selasa	09.30	120	TF4212		KALKULUS II		P.707
Selasa	09.30	120	TF4223	B	ALJABAR LINIER DAN MATRIKS	Ir. KARTIKA GUNADI, M.T.	P.622A

Budhi GS, Handojo A, Soloment B, (2009), Pemanfaatan Compact Genetic Algorithm (cGA) Untuk Optimasi Penjadwalan Penggunaan Ruang Kuliah Di U.K. Petra, Proceeding Seminar Nasional Aplikasi Teknologi Informasi 2009, Yogyakarta, 20 Juni 2009

Aplication Examples

Course Scheduling for Students during the PRS using a Genetic Algorithm Method





Jadwal Kuliah

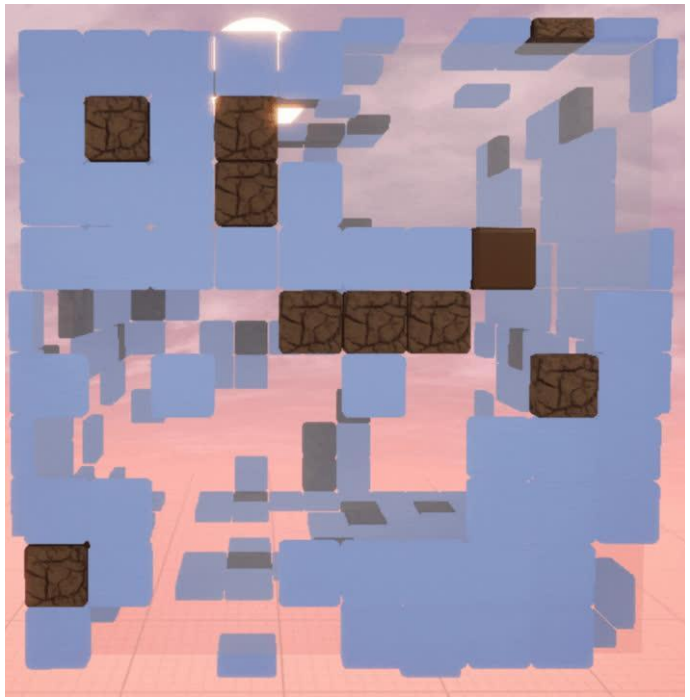
No	Hari	Mulai	Selesai	Nama Matkul	Kelas
1	Senin	17:30	19:30	Kal 1 - Kuliah	D
2	Selasa	17:30	19:30	Kal 1 - Responsi	D
3	Kamis	13:30	15:30	PBO - Kuliah	B
4	Jumat	08:30	10:30	KDJK - Kuliah	-
5	Jumat	14:30	17:30	PAB Web - Kuliah	-
6	Sabtu	07:30	10:30	ERP - Kuliah	-
7	Sabtu	10:30	13:30	ADSI - Kuliah	-

			elesai	Nama Matkul	Kelas
			6:30	ERP	-
			9:30	PAB Web	-
			0:30	ADSI	-
			16:30	KDJK	-
5	Ke - 6	07:30	10:30	PBO	B
6	Ke - 7	13:30	16:30	Kal 1	D

Budhi GS, Gunadi K, Wibowo DA, (2015), Genetic Algorithm for Scheduling Courses, In Intelligence in the Era of Big Data, ICSIIT 2015, Communications in Computer and Information Science, vol 516, Springer, Berlin, Heidelberg.

Aplication Examples

Procedural Content Generation Of Maps In A 3D Puzzle Game Using Genetic Algorithm (Nicholas Soetikno, C14210110)



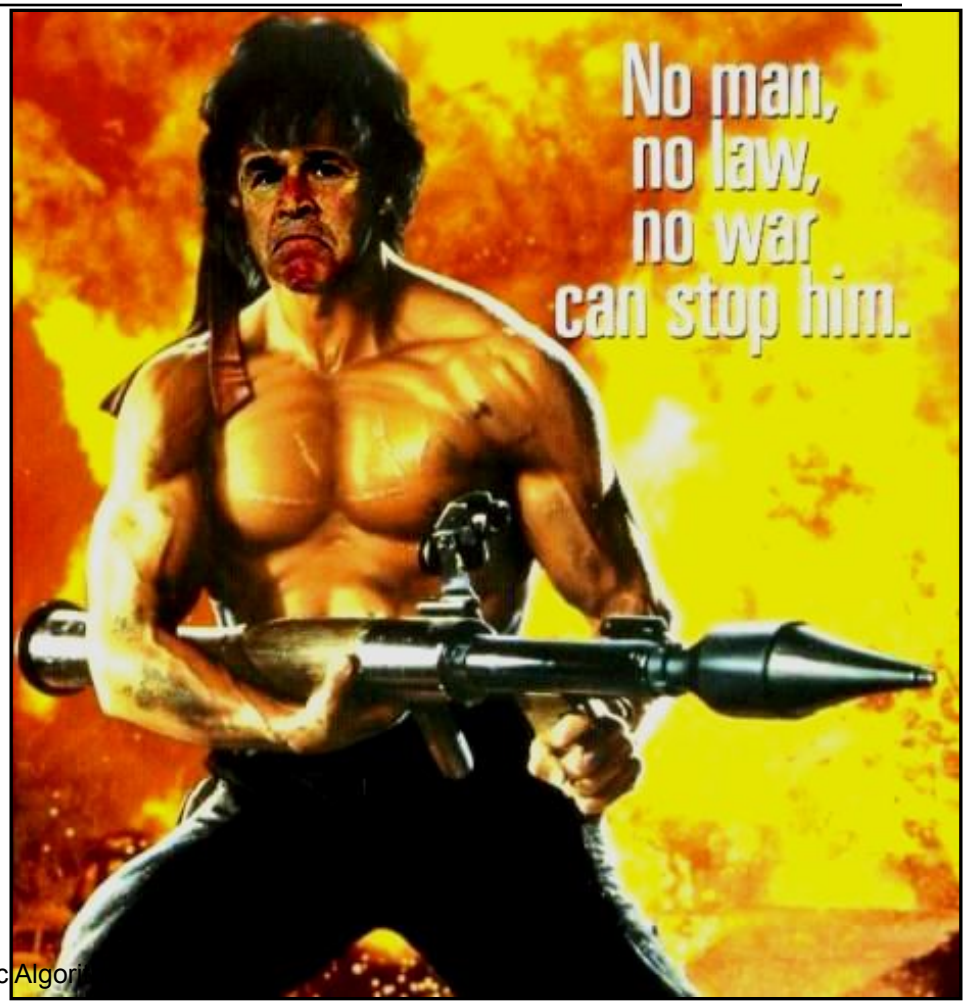
Biological Background

- **Chromosomes:** strings of DNA and serves as a model for the organism.
- The chromosomes consist of **genes** (blocks of DNA).
  - each has encoded a trait / attributes of the organism (eye color, hair color, height, etc).
  - **Alleles:** The possible settings for a trait.
  - **Locus:** The position of a gene in the chromosome.
- **Genome:** The complete set of the all chromosomes.
- **Genotype:** The set of genes in a genome.

Biological Background

Reproduction:

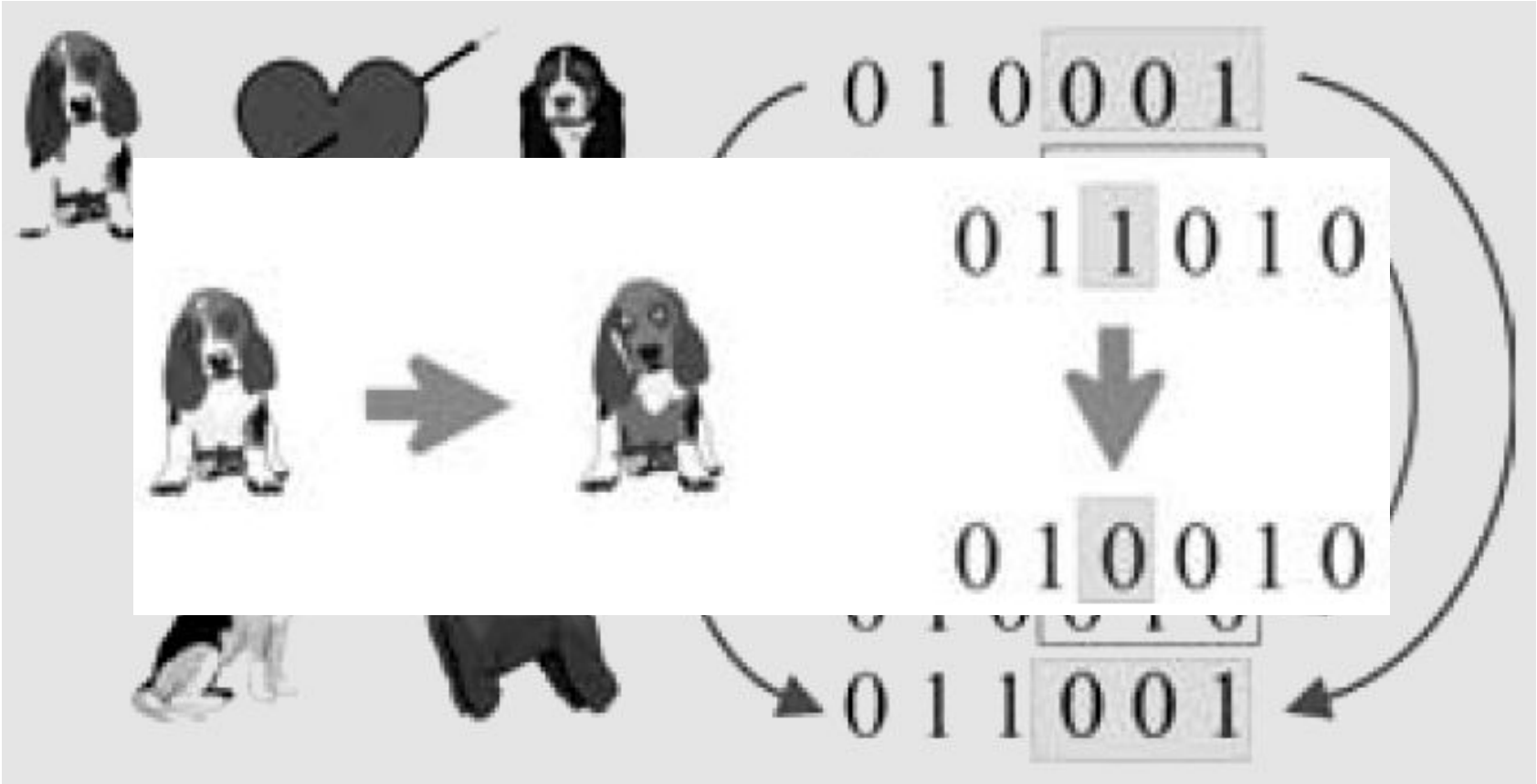
- **Crossover** ➪ When the parents "meet" to form a whole new chromosome.
- The new offspring can be mutated.
  - **Mutation** causes the elements of the DNA to be changed slightly.
  - This change is mainly caused by errors in copying genes from parents.
- **The Fitness** of an organism is determined by the success of an organism in its life.



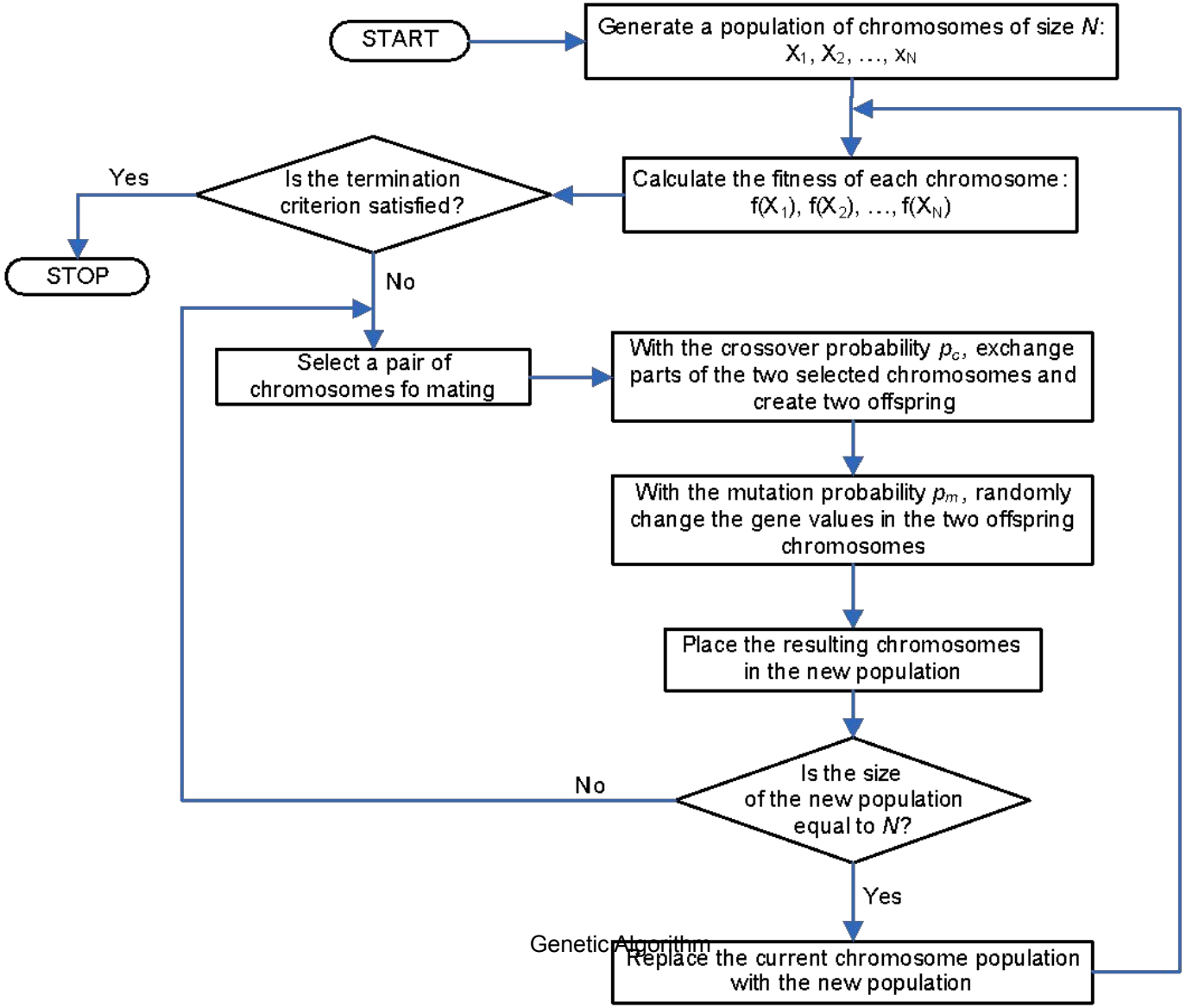
GA Basic Description

- **Solution:**
  - **Chromosome:** The array of **a possible solution**
  - **Gene:** **An element** in the array of solution
- **Chromosomes** are broken up and recombined using genetic operators (**crossover** and **mutation**) to create (an often better) new solution.
- **A population:** A set of possible solution those are generated and evaluated.

Analogy between GA and biological genetics



The Basic Genetic Algorithm (Flowchart)



The Basic Genetic Algorithm

1. Generate a population of chromosomes of size N: $X_1, X_2, \ldots, X_N$
2. Calculate the fitness of each chromosome: $f(X_1), f(X_2), \ldots, f(X_N)$
3. Is the termination criterion satisfied? **If** True **then** **STOP** **else do** **Step 4 to 8**
   4. Select a pair of chromosomes fo mating
   5. With the crossover probability p_c, exchange parts of the two selected chromosomes and create two offspring
   6. With the mutation probability p_m, randomly change the gene values in the two offspring chromosomes
   7. Place the resulting chromosomes in the new population
   8. Is the size of the new population equal to N? If False **then go to** **Step 4** **else continue to** **the next step**.
9. Replace the current chromosome population with the new population then **go to** **Step 2**.

Encoding Operators

- Encoding the chromosome is **the first big problem** encountered in a genetic algorithm.
- The encoding will be dependent on the type of problem.

- **Binary Encoding:**

Chromosome A	**101100101100101011100101**
Chromosome B	**111111100000110000011111**

  - Every chromosome is a string of bits 0/1.
  - Gives many possible chromosomes even with a small number of alleles.
  - Not natural for many problems.

Encoding Operators

- **Permutation Encoding:**

Chromosome A	1	5	3	2	6	4	7	9	8
Chromosome B	8	5	6	7	2	3	1	4	9

  - Usually used in ordering problems, such as Travelling Salesman Problem or Task Ordering Problem.
  - Every chromosome is **a string of numbers**, which represents number in **a sequence**.
  - Corrections must be made crossover and mutation to leave the chromosome consistent.

Encoding Operators

- **Direct Value Encoding:**

Chromosome 1	1.2324 ; 5.3243 ; 0.4556 ; 2.3293 ; 2.4545
Chromosome A	ABDJEIFJDHDIERJFDLDFLFEGT
Chromosome Move1	(back), (back), (right), (forward), (left)

  - Direct value encoding can be used in problems, where some complicated value, such as real numbers, are used.
  - Every chromosome is a string of some values.
  - Values can be anything connected to problem, form numbers, real numbers or chars to some complicated objects.
  - Very good for some special problems.
  - Often necessary to **develop new types of crossover and mutation** specific for the problem.

Chromosome Model Example

Classroom schedulling in U.K. Petra using compact GA
(Billy Soloment / 26404012)

Novak	DI4213A	DV4212B	SI4220A	TI4215C
Ruang				
Hari	1	1	1	1
Jam	7:30	7:30	8:30	9:30
Lama	60	120	60	180
FCNIP	02-123	00-101	01-121	0

- Notes:
  - Novak : Course Code
  - Ruang : Classroom
  - Hari : Day the course runs
  - Waktu : Time the course begin
  - Lama : Duration of the course
  - FC-NIP : Lecturer Status (Fulltime or partime lecturer)

* The GA process is actually only used to organize the classroom, while other attributes are used to support the calculation of fitness costs.

Chromosome Model Example

Course Scheduling for Students during the PRS using a Genetic Algorithm Method (Denny Alexander Wibowo / 26408040)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
hari - 0	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
hari - 1	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
hari - 2	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
hari - 3	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
hari - 4	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
hari - 5	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
hari - 0	[2]	[2]	[2]	[0]	[0]	[6]	[6]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]
hari - 1	[0]	[0]	[49]	[49]	[0]	[0]	[0]	[0]	[10]	[10]	[0]	[0]	[0]	[0]	[0]
hari - 2	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]
hari - 3	[0]	[0]	[0]	[0]	[0]	[0]	[19]	[19]	[19]	[0]	[0]	[0]	[0]	[0]	[0]
hari - 4	[0]	[33]	[33]	[33]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]
hari - 5	[0]	[0]	[0]	[0]	[44]	[44]	[44]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]

ID kelas	49
Nama mata kuliah	Struktur Data
Kelas	A
SKSk	2
SKSp	1
SKSs	0
SKSr	0
Syarat SKS	0
Syarat mata kuliah	Lulus PBO, Lulus AP
Jam mulai	08.30
Jam selesai	10.30
Jadwal ujian	Hari ke 3, jam 10.30
Kuliah peminatan	Bukan

Chromosome Model Example

Ready-mix Concrete Delivery Scheduling Optimization **(Vince Pongawa)**

Gene:

Attribut	Type	Size	Keterangan
tanggal_kirim	char	20	Untuk menyimpan tanggal pengiriman
jam_kirim	char	20	Untuk menyimpan waktu truk mulai diisi untuk dikirimkan
jam_selesai_isi	char	20	Untuk menyimpan waktu truk selesai diisi
jam_kirim_lagi	char	20	Untuk menyimpan waktu *available* untuk pengiriman berikutnya
jam_kembali_plant	char	20	Untuk menyimpan waktu truk kembali ke *plant*
truk	char	20	Untuk menyimpan truk yang digunakan untuk mengirim
proyek	char	20	Untuk menyimpan nama proyek yang dikirim
jumlah_kirim	integer	5	Untuk menyimpan jumlah pengiriman (dalam m^3)
sisa	integer	5	Untuk menyimpan sisa pengiriman yang belum dikirim (dalam m^3)
status	integer	1	Untuk menyimpan status pengiriman apakah sedang dikirim atau telah selesai

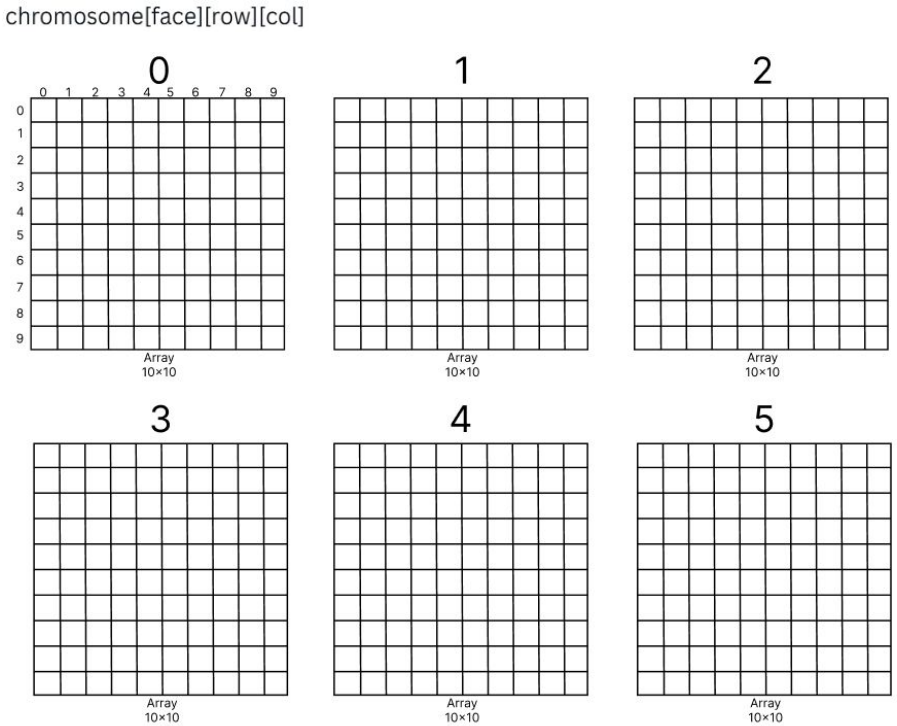
Chromosome:

CHROMOSOME REPRESENTATION

Procedural Content Generation Of Maps In A 3D Puzzle Game Using Genetic Algorithm (Nicholas Soetikno, C14210110)

Each cell within the 10x10 grids stores an "element_id", the types of element_ids are:

- **0:** Path (empty tile for player movement)
- **1:** Wall (regular obstacle blocking paths)
- **2:** CrumbleWall (fragile wall that collapses after interaction)
- **3:** Switch (interactive object linked to a door)
- **4:** Door (object that opens/closes, controlled by a switch)
- **5:** MovingBox (object that moves with gravity, used for traversal)
- **6:** Goal (end point of the level)
- **7:** Spawn (player's starting point)

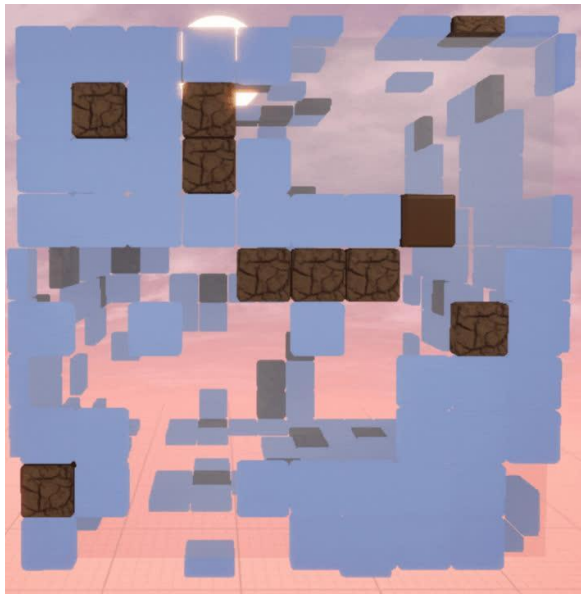


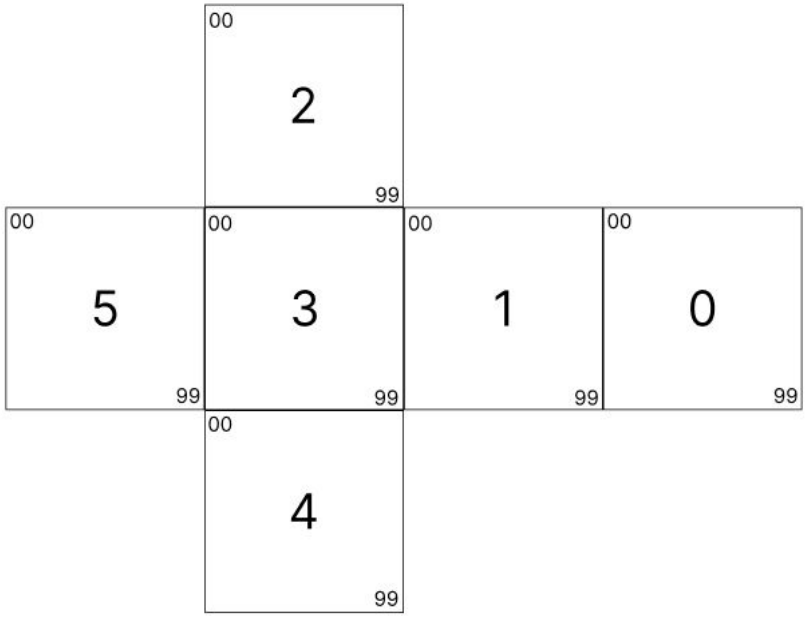
CUBE FACE CONNECTIVITY

Procedural Content Generation Of Maps In A 3D Puzzle Game Using Genetic Algorithm (Nicholas Soetikno, C14210110)

- Each of the six 10x10 2D grids are connected to each other, each representing a face of the cube.

- When a player moves off the edge of one cube face (e.g., from Face A to Face C), they transition to the adjacent face.





Genetic Operators

Crossover:

- Selects from the parent chromosomes to make the offspring.
- **Example:** Single Point Crossover
  - Randomly choosing a crossover point, copy up to the point from one parent, then copy the rest from the other parent.

- Crossovers difficulty depending on the type of encoding used in the chromosome.

Genetic Operators

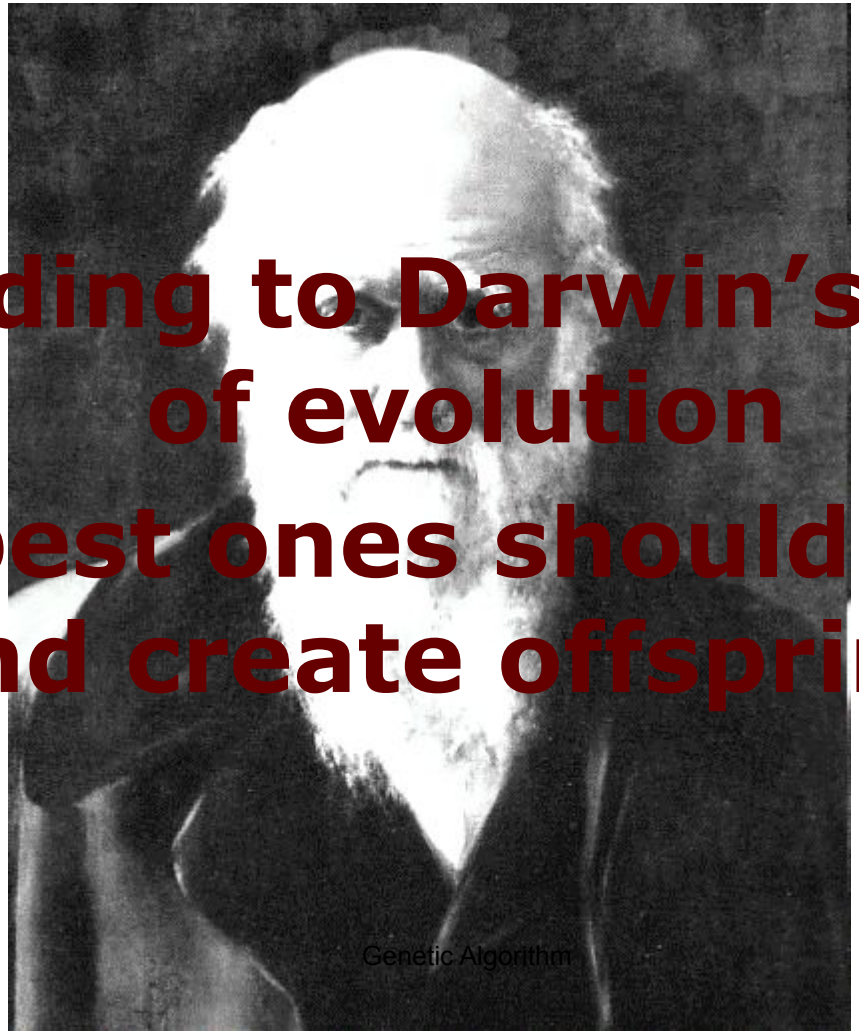
Mutation:

- To prevent putting all the solution in a population into a local optimum.
- **Example:** Bit Inversion
  - The mutation operator randomly changes parts of the chromosome of the new offspring.

Original offspring 1	110**1**1110000011110
Original offspring 2	1101100100110110

- The mutation is dependent upon the way the chromosome is encoded.

Selection Operators



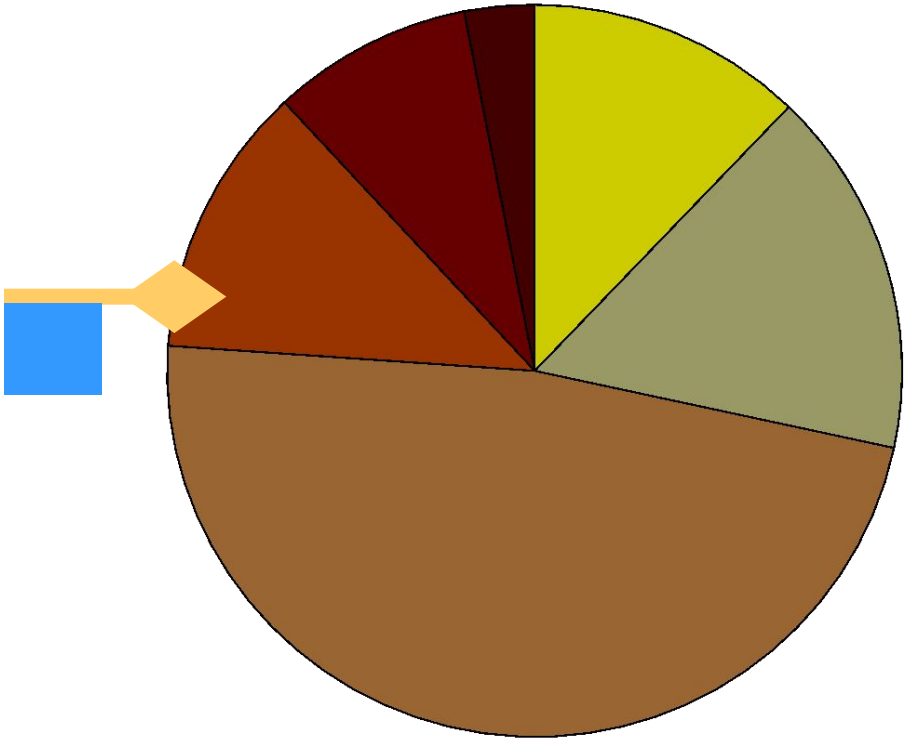
According to Darwin's theory of evolution

"The best ones should survive and create offspring"

Selection Operators

Roulette Wheel Selection:

- The chromosomes are selected according to their fitness.
- The better the chromosomes are, the more chances to be selected.
- The Population is a **roulette wheel** where all chromosomes are placed according to its fitness.



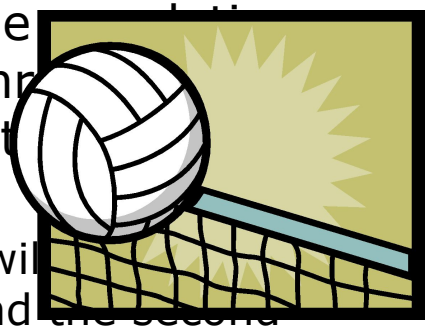
Selection Operators

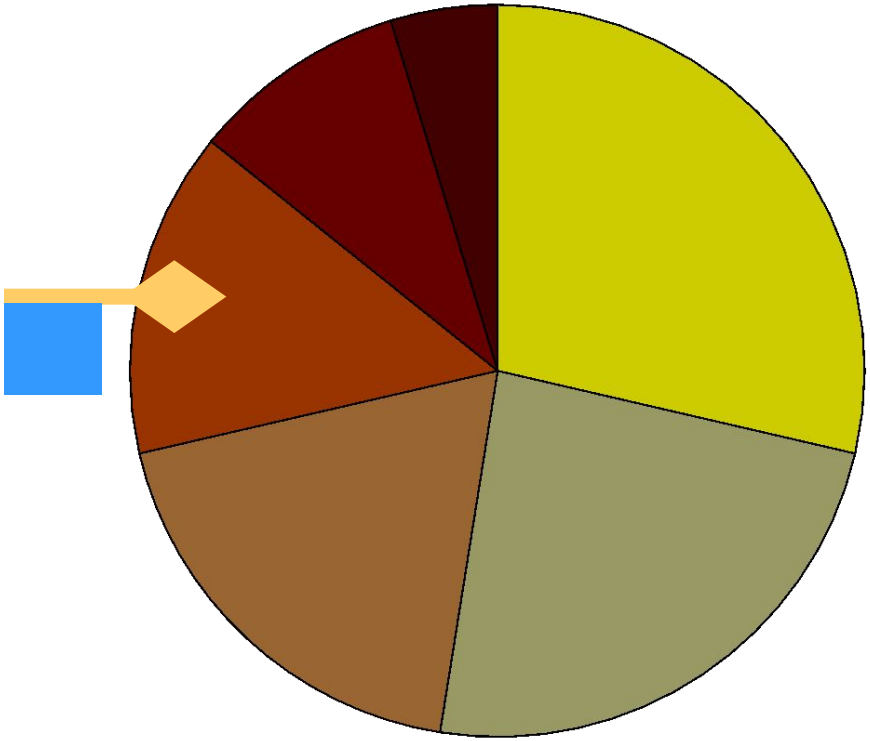
Roulette Wheel Selection Algorithm:

1. Calculate the sum of all chromosome fitnesses in the population ☐ sum **S.**
2. Generate random number from interval ***(0, S)*** ☐ ***r.***
3. Go through the population and sum fitnesses from ***0*** - sum ***S***. When the sum ***S*** is greater then ***r***, stop and return the chromosome where you are.

Selection Operators

Rank Selection:





ave a fitness 2

- The best will have a fitness

Selection Operators

Steady State Selection:

- Not really a good method of selection.
- The main idea:
  - A good part of the chromosomes should survive to the next generation.
  - In every generation a few with high fitness for creating offspring,
  - Some with low fitness are removed and the new offspring put in its place.
- The rest of the population survives to the next generation.

Selection Operators

Elitism / Trun

"When creati
by crossover
is a chance t
best chromos

- **The best**
chromosom
new popula
- The rest don
  - **Roulette**
  - **Rank Sel**

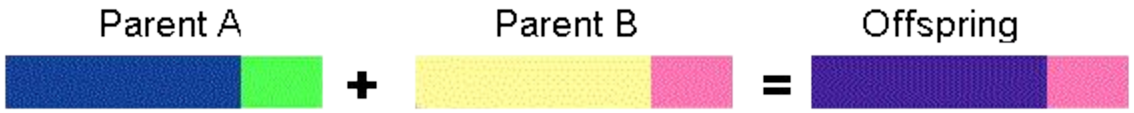
- This method
the optimal
algorithm by
found solutio

Crossover and Mutation

- The type and implementation of the operators are dependent on the type of encoding used in the algorithm.

- **Binary Encoding:**
  - **Crossover**

    1. Single point crossover:

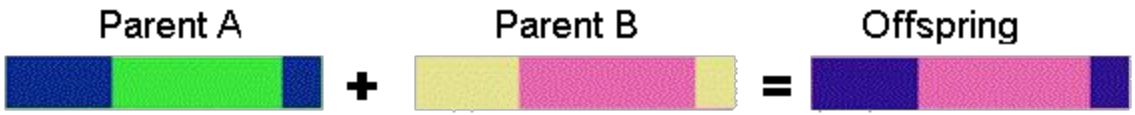


11001011 + 11011111 = 11001111

Crossover and Mutation

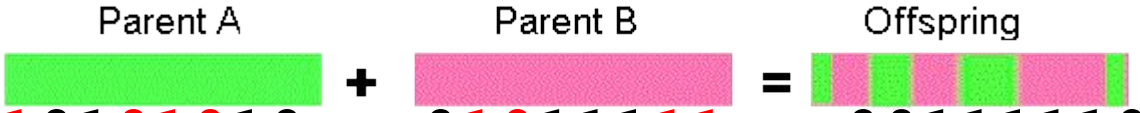
- **Binary Encoding:**
  - **Crossover**

    2. Two point crossover:



11001010 + 11011111 = 11011110

3. Uniform crossover - bits are randomly copied from the first or from the second parent.



10101010 + 01011111 = 00111110

Crossover and Mutation

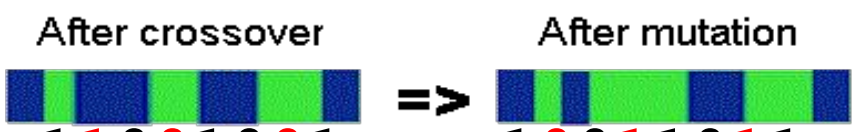
- **Binary Encoding:**
  - **Crossover**

    4. Arithmetic crossover - some arithmetic operation is performed to make a new offspring.

11001001 + 11011111 = 11001001 (AND)

  - **Mutation**

    Bit inversion - selected bits are inverted.



11001001 => 10011011

Crossover and Mutation

- **Permutation Encoding:**
  - **Crossover**

    Single point crossover - one crossover point is selected, up till this point the permutation is copied from the first parent, then the second parent is scanned and if the number is not yet in the offspring it is added.

    (1 2 3 4 5 6 7 8 9) + (4 5 3 6 8 9 7 2 1)

    Offspring 1 = (1 2 3 4 5 *6 8 9 7*)

    Offspring 2 = (*3 4 5 6 8* 9 7 2 1)

  - **Mutation**

    Order changing - two numbers are selected and exchanged.

    (1 2 3 4 5 6 8 9 7) => (1 8 3 4 5 6 2 9 7)

Crossover and Mutation

- **Direct Value Encoding:**
  - **Crossover**

    All crossovers from binary encoding can be used!!

  - **Mutation**

    Adding / Subtracting (for real value encoding):

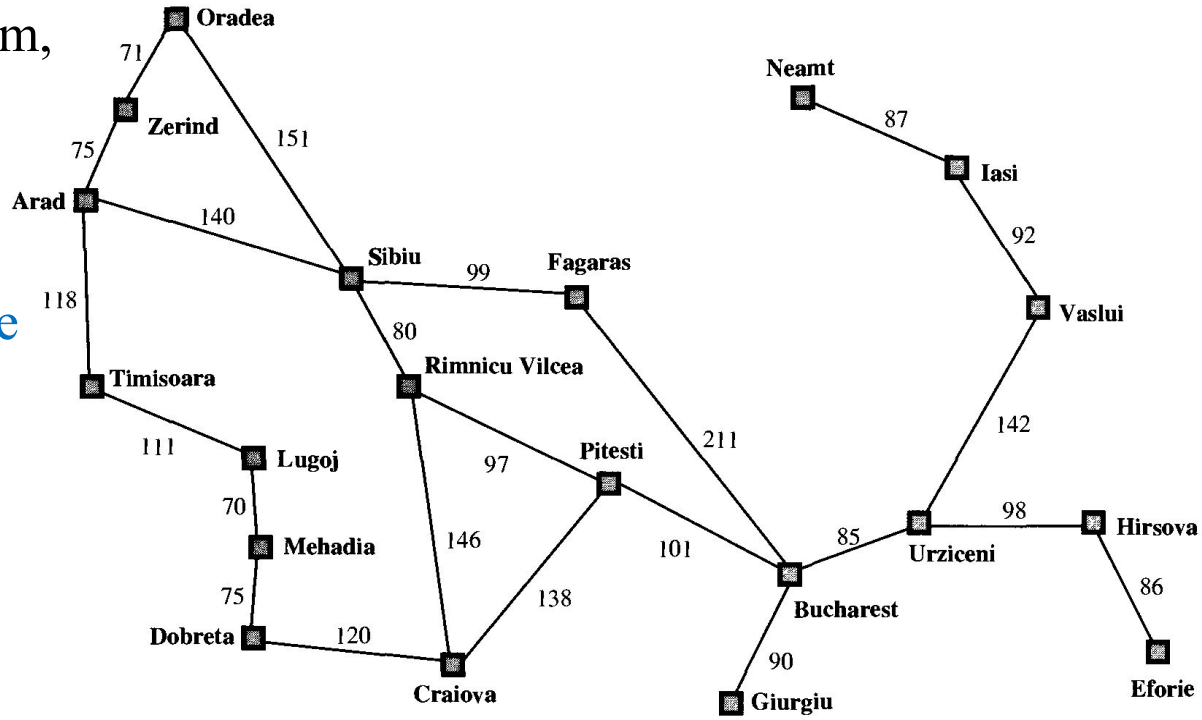
    - Selected values is added (or subtracted) by a small number

```
(1.29  5.68  2.86  4.11  5.55) =>
(1.29  5.68  2.73  4.22  5.55)
```

Fitness Cost

- The formula is created based on the problem
- E.g., Pathfinding problem,

Fitness cost = the cost of getting from the initial state to the goal state.



Fitness Cost

- Ready-mix Concrete Delivery Scheduling Optimization (Vince Pongawa)
  - Fitness Cost is calculated based on three priorities:
    1. **Number of Projects Completed** in one day.
    2. **Remaining Free Time** in 1 day, calculated based on free time in 1 day (7:30 to 7:29 the next day), starting from the arrival time of the last truck until 7:29 the next day.
    3. **Total Remaining Delivery**, calculated based on the remaining delivery per project.
  - **Fitness Cost** = Number of Projects Completed * 1000 +
    Remaining Free Time * 100 +
    Total Remaining Delivery * 50.

A Simple Example

- **The Traveling Salesman Problem:**
  - Find a tour of a given set of cities so that:
    - each city is visited only once
    - the total distance traveled is minimized

- **Representation:**
  - Representation is an ordered list of city numbers known as an **Order-based GA**.

A) London	B) Beijing	C) Tokyo
D) Venice	E) Singapore	F) Sydney

A Simple Example

- **Create a random population**

```
CityList1 (A B C D E F)
CityList2 (B E D F A C)
CityList3 (B A D C F E)
CityList4 (F A E B D C)
```

- **Crossover =** *Single Point (2-3)*
- **Mutation =** *Order Changing*
- **Selection =** *Roulette Wheel*
- **Max Pop =** *4*
- *Fitness Cost?*

Distances Matrix

	A	B	C	D	E	F
A	0	5	3	5	1	6
B	5	0	2	6	9	3
C	3	2	0	8	4	6
D	5	6	8	0	6	4
E	1	9	4	6	0	3
F	6	3	6	4	3	0

A Simple Example

- **Create a random population**

  `CityList1 (A B C D E F)`

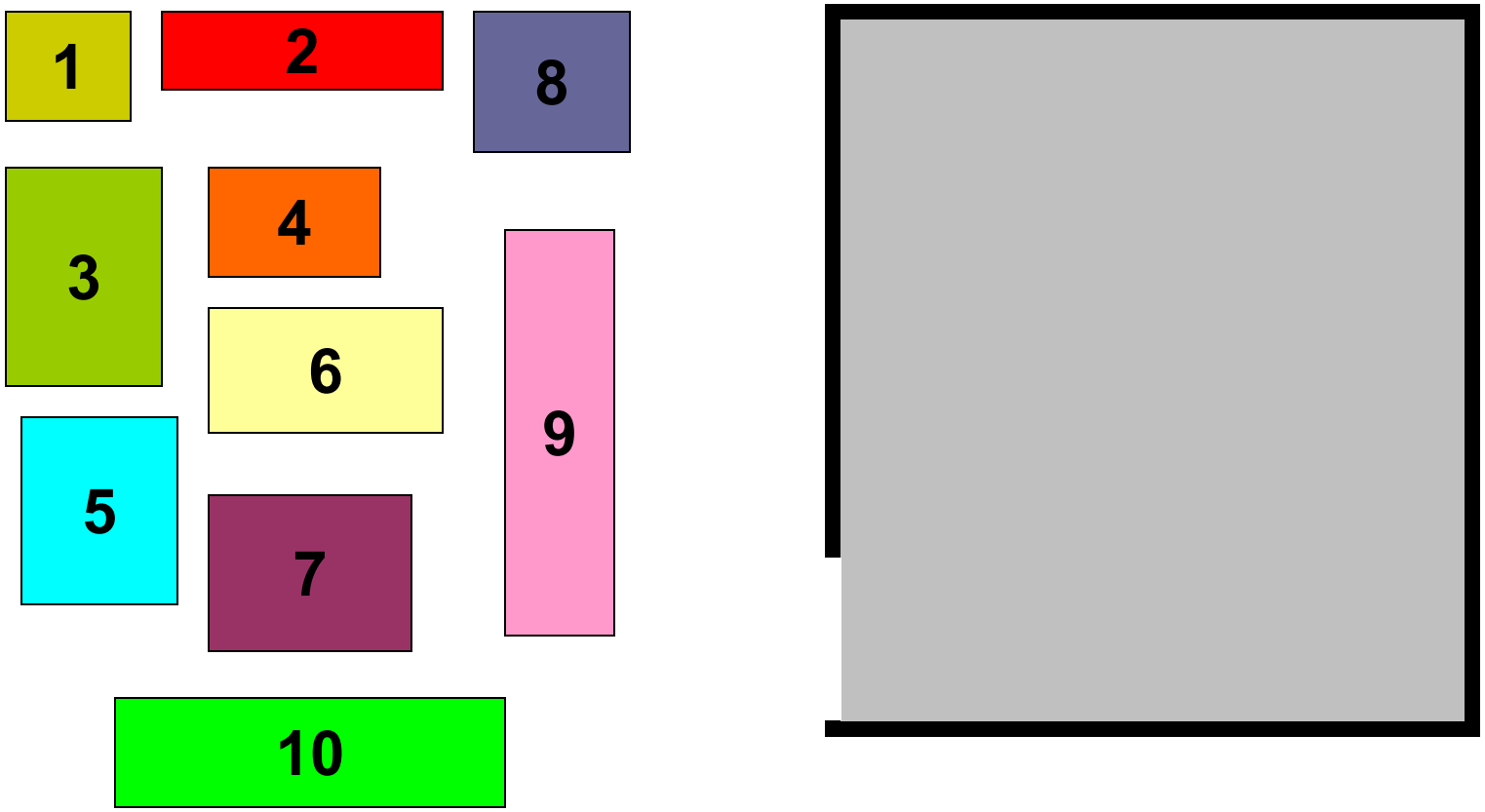
  `CityList2 (B E D F A C)`

- **Crossover =** *Single Point (3-4)*
- **Mutation =** *Order Changing*
- **Selection =** *Steady State*
- **Max Pop =** *4*
- *Fitness Cost?*

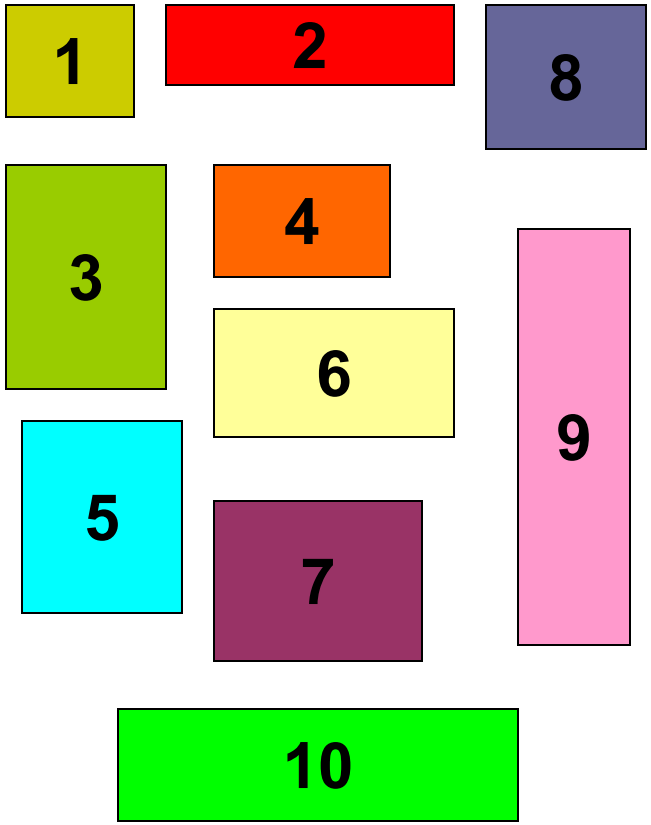
Distances Matrix

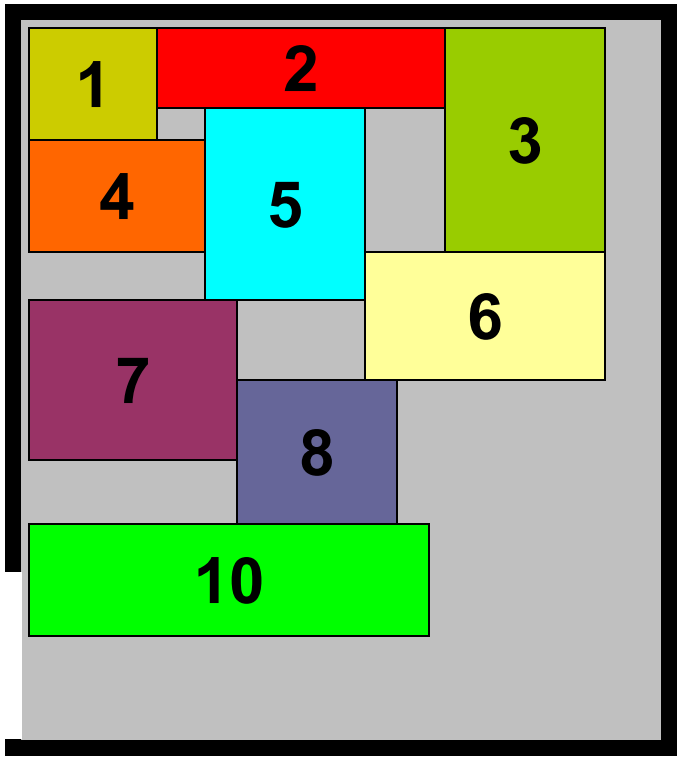
	A	B	C	D	E	F
A	0	5	3	5	1	6
B	5	0	2	6	9	3
C	3	2	0	8	4	6
D	5	6	8	0	6	4
E	1	9	4	6	0	3
F	6	3	6	4	3	0

Example (Simple Warehouse Optimizer)



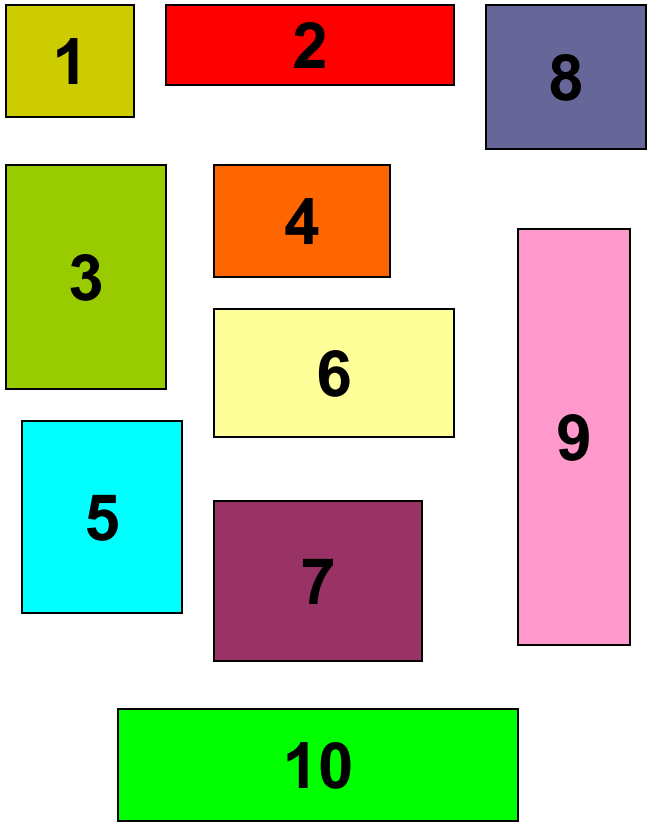
Example (Simple Warehouse Optimizer)

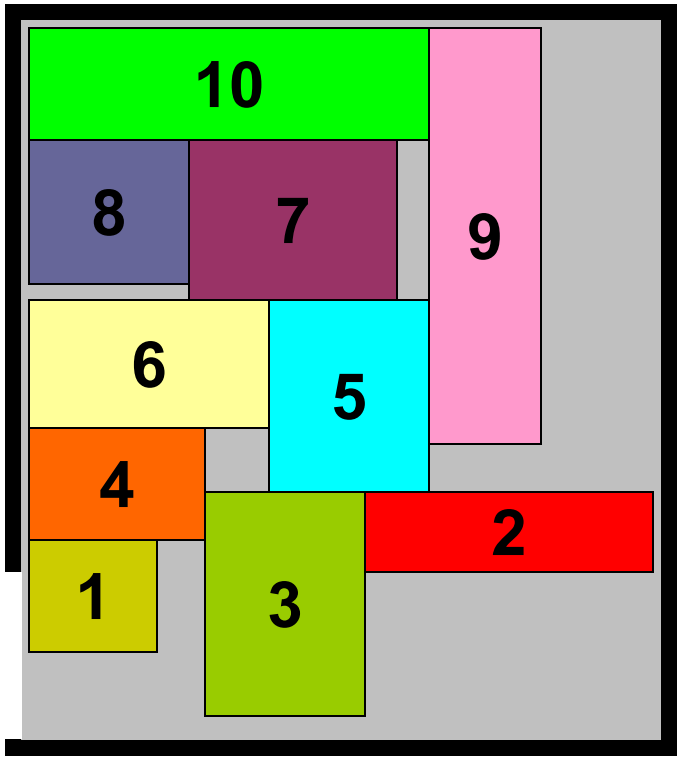




1st Alternative

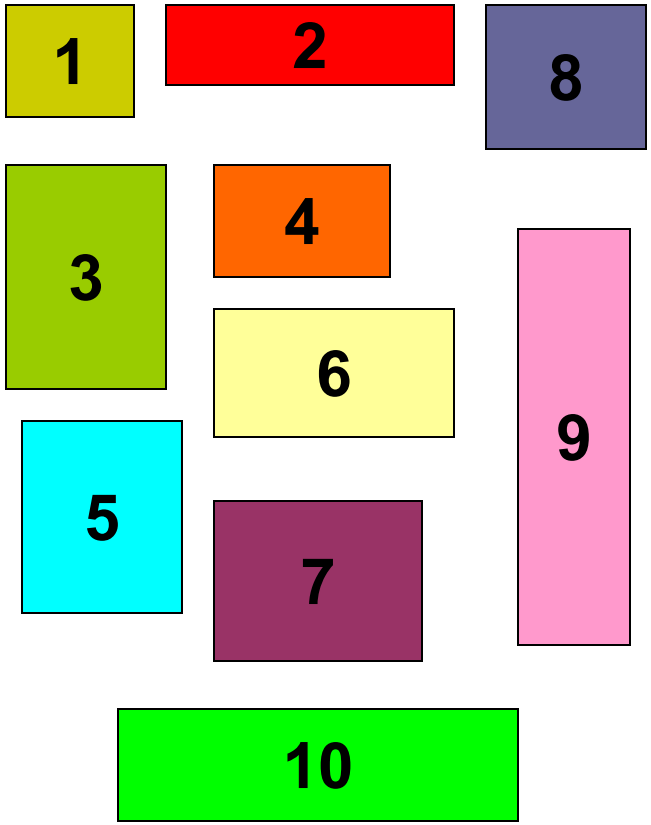
Example (Simple Warehouse Optimizer)

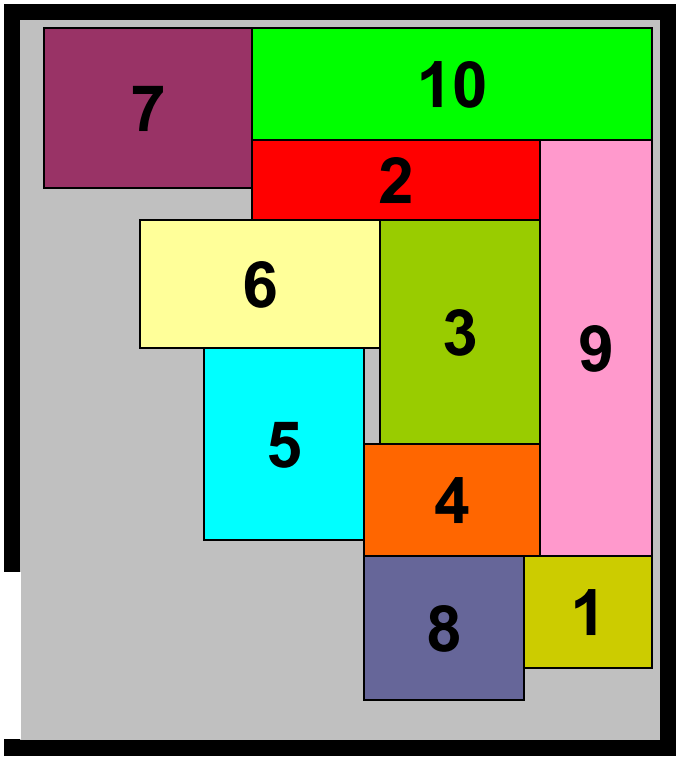




2nd Alternative

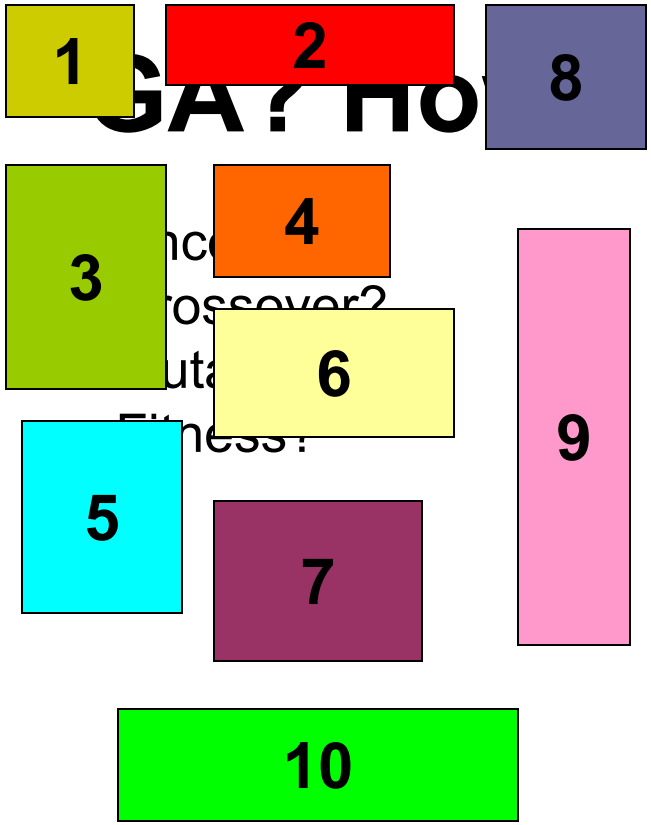
Example (Simple Warehouse Optimizer)

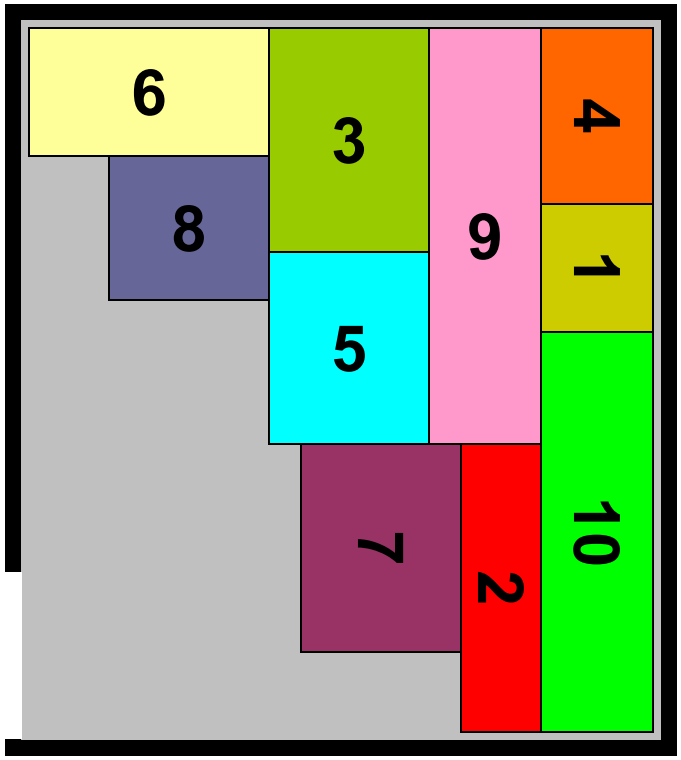




3rd Alternative

Example (Simple Warehouse Optimizer)





4th Alternative

Exercise 1:

- **Create a random population**

  `CityList1 (A B C D E F)`

  `CityList2 (B E D F A C)`

- **Crossover =** *Single Point (3-4)*
- **Mutation =** *Order Changing*
- **Selection =** *Steady State*
- **Max Pop =** *5*
- *Fitness Cost?*
- *Run it for 4 iterations (point: 2 for each iteration)*

Distances Matrix

	A	B	C	D	E	F
A	0	5	3	5	1	6
B	5	0	2	6	9	3
C	3	2	0	8	4	6
D	5	6	8	0	6	4
E	1	9	4	6	0	3
F	6	3	6	4	3	0

Erxercise 2:

- **Create a random population**

```
CityList1 (A B C D E F)
CityList2 (B E D F A C)
CityList1 (B A D C F E)
CityList2 (F A E B D C)
```

- **Crossover =** *Single Point (1-2)*
- **Mutation =** *Order Changing*
- **Selection =** *Roulette Wheel*
- **Max Pop =** *4*
- *Fitness Cost?*
- *Run it for 4 iterations (point: 2 for each iteration)*

Distances Matrix

	A	B	C	D	E	F
A	0	5	3	5	1	6
B	5	0	2	6	9	3
C	3	2	0	8	4	6
D	5	6	8	0	6	4
E	1	9	4	6	0	3
F	6	3	6	4	3	0

Exercise 3:

- **Create a random population**

  `CityList1 (A B C D E F)`

  `CityList2 (B E D F A C)`

- **Crossover =** *Two Point (1-2 & 3-4)*
- **Mutation =** *Order Changing*
- **Selection =** *Steady State*
- **Max Pop =** *4*
- *Fitness Cost?*
- *Run it for 4 iterations (point: 2 for each iteration)*

Distances Matrix

	A	B	C	D	E	F
A	0	5	3	5	1	6
B	5	0	2	6	9	3
C	3	2	0	8	4	6
D	5	6	8	0	6	4
E	1	9	4	6	0	3
F	6	3	6	4	3	0

Erxercise 4:

- **Create a random population**

```
CityList1 (A B C D E F)
CityList2 (B E D F A C)
CityList1 (B A D C F E)
CityList2 (F A E B D C)
```

- **Crossover =** *Two Point (2-3)*
- **Mutation =** *Order Changing*
- **Selection =** *Elitsm*
- **Max Pop =** *4*
- *Fitness Cost?*
- *Run it for 4 iterations (point: 2 for each iteration)*

Distances Matrix

	A	B	C	D	E	F
A	0	5	3	5	1	6
B	5	0	2	6	9	3
C	3	2	0	8	4	6
D	5	6	8	0	6	4
E	1	9	4	6	0	3
F	6	3	6	4	3	0